

RecipEZ: Architecture & Documentation

Michael Hannan | 3D Recipe Search Engine Pipeline

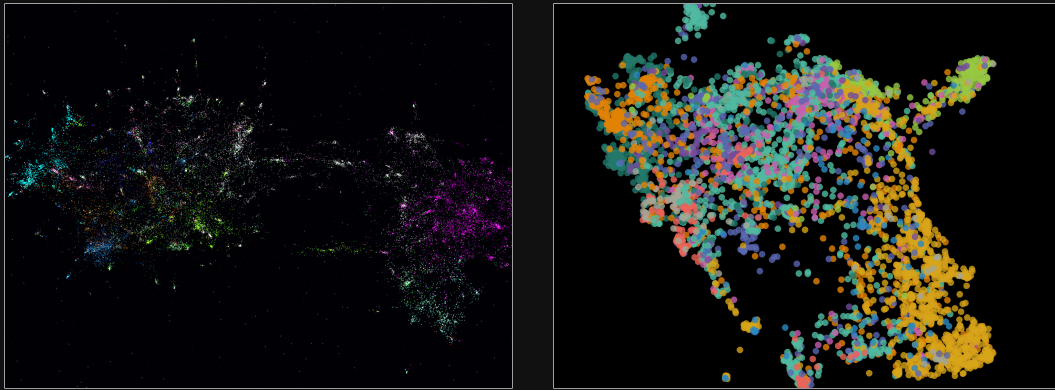


Figure 1: Comparison between V2.1 and V1.3

1. PREFACE

Wandering on the modern web is such a major nuisance. Scouring the web you can find bloat covered ads about practically any topic, everything is built to either gather data or make money off of you. RecipEZ was intended to be a fun personal project that helped solve this problem in the context of cooking and specifically recipes. Recipes tend to be a type of content that falls heavily for this bloat trap, with ads for cookware, meal kits, and grocery delivery services littering the search results and life stories about authors that have nothing to do with the actual recipe. The goal of RecipEZ was to create a simple, interactive search engine that accepts a list of raw ingredients and outputs the top recipe matches in a visually appealing 3D space.

2. BRAINSTORMING AND ARCHITECTURE

With the constraints defined—accepting a list of raw ingredients and outputting top recipe matches—several approaches were evaluated to establish a mathematical or semantic relationship between the input and the output.

2.1. Initial Brainstorm

The immediate thought was to simply pass the ingredients into a Large Language Model (LLM) and have it generate a recipe. However, this approach was discarded as it is computationally

expensive, abstracts away the core technical challenge of building a search engine, and introduces hallucination risks. Fine-tuned Sequence-to-Sequence (Seq2Seq) models were also evaluated. While excellent for machine translation, they require massive amounts of training data and compute resources that were out of scope for this architecture.

2.2. Data

One of the biggest challenges was dealing with data. The initial dataset was riddled with inconsistencies and was not considered normalized. For example, the same ingredient could be represented in multiple ways (“chopped onions”, “onions, chopped”, “3/4 cups of onions”). Because the vectorization process treats unique text strings as separate features, these semantic equivalents were corrupting the vector distance calculations. After several attempts to write custom cleaning algorithms, the most effective solution was pivoting to an entirely separate, pre-normalized dataset to ensure clean vectorization.

2.3. Model Selection

When initially researching I had come across multiple different techniques to search for recipes through ingredients. There are two main approaches: classification or clustering. Classification would involve training a model to predict the recipe category based on the ingredients, while clustering groups similar recipes together in a high-dimensional space. The clustering approach was chosen because the goal is to filter and find recipes similar to the input ingredients, allowing for a more flexible and intuitive search experience. This led to the selection of TF-IDF for vectorization, UMAP for dimensionality reduction, and K-NN for searching. SBERT was later explored as an alternative vectorization strategy.

3. THE DATA PIPELINE

The core engine of RecipEZ operates in three distinct phases to take text and turn it into a 3D searchable galaxy.

PIPELINE FLOW: INPUT → [TF-IDF](#) → [SVD](#) → [UMAP](#) → [K-NN](#) → [THREE.JS](#)

The full pipeline transformation across all stages can be traced visually in [Figure 6](#) , which shows how individual recipes from each culinary category are reshaped at every step from raw ingredient counts through to final UMAP coordinates.

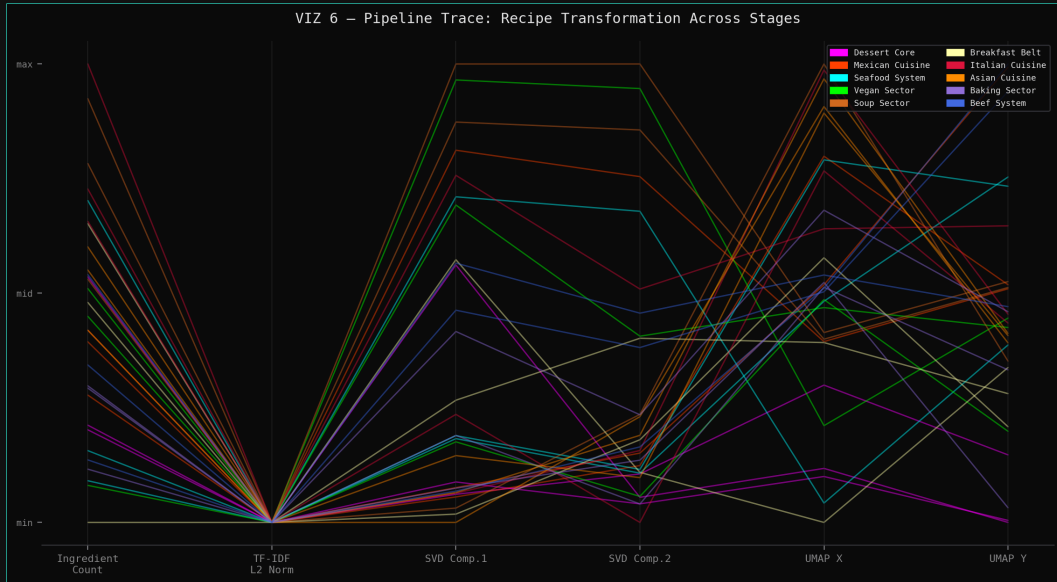


Figure 2: [VIZ 6](#) — Pipeline Trace: parallel coordinates showing recipe transformation across all stages. Each polyline is one recipe; colors encode culinary category. The convergence from chaotic left-side axes to structured right-side UMAP coordinates demonstrates the pipeline collapsing high-dimensional noise into geometry.

3.1. Vectorization

The first step converts textual data—ingredients and tags—into a numerical format that machine learning algorithms can process. [TF-IDF](#) weighs the importance of each word in the context of the entire dataset. Later iterations also explored SBERT, an advanced method using pre-trained language models to generate dense vectors that capture deeper semantic relationships.

The key mechanism TF-IDF exploits is the inverse document frequency penalty: common ingredients like “salt” or “water” that appear in nearly every recipe are mathematically suppressed, while rare and distinctive ingredients like “saffron” or “za’atar” receive high weight. The full spectrum of this penalty across the vocabulary is shown in [Figure 1](#).

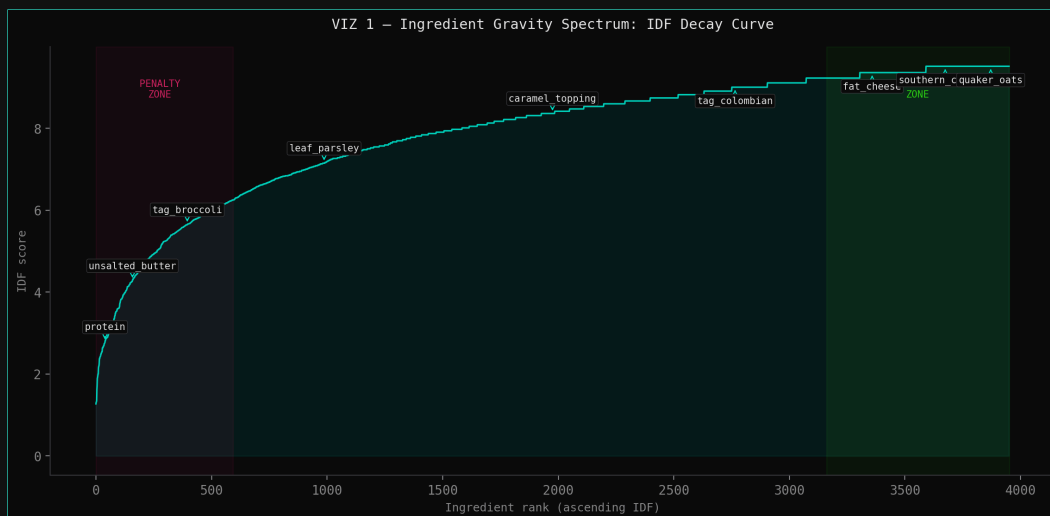


Figure 3: [VIZ 1](#) — Ingredient Gravity Spectrum: IDF decay curve sorted by ascending score. The shaded pink zone on the left marks pantry staples penalized to near-zero. The green zone on the right marks high-IDF ingredients that carry the most discriminative weight in the vector space. Annotated words reveal which specific ingredients fall at key inflection points.

3.2. Dimensionality Reduction

After vectorization, the data exists in an extremely high-dimensional space (up to 10,000 dimensions). To render this in a web browser, the dimensions must be reduced while preserving local and global structure. [SVD](#) is used first to discard noise and identify latent features, compressing the sparse TF-IDF matrix into 100 dense latent components. Following this, [UMAP](#) compresses the data into a 3D coordinate system, creating the clusters visualized in the final application.

3.3. Clustering and Searching

With recipes mapped in 3D space, the search engine functionality is powered by [K-NN](#). When a user inputs ingredients, the text is vectorized, placed into the space, and the algorithm retrieves the closest recipe nodes using cosine similarity, which are then passed to the frontend.

[Figure 4](#) shows a live query dissection: a concrete set of input ingredients is vectorized, projected into 2D UMAP space, and the Top-8 returned recipes are annotated with their names and cosine similarity scores.



Figure 4: [VIZ 4](#) — Search Oracle: K-NN query anatomy for the query “eggs, bacon, cheddar, sourdough, butter.” The teal star marks the projected query vector. Concentric dashed rings represent cosine distance thresholds. Pink highlighted points are the Top-8 returned recipes; each is annotated with its recipe name and cosine similarity score. The spatial clustering of results confirms that K-NN is recovering semantically coherent neighborhoods.

To validate that the spatial clusters in the 3D galaxy correspond to real culinary categories, [Figure 5](#) cross-tabulates K-Means spatial clusters against the Tag Ontology labels.

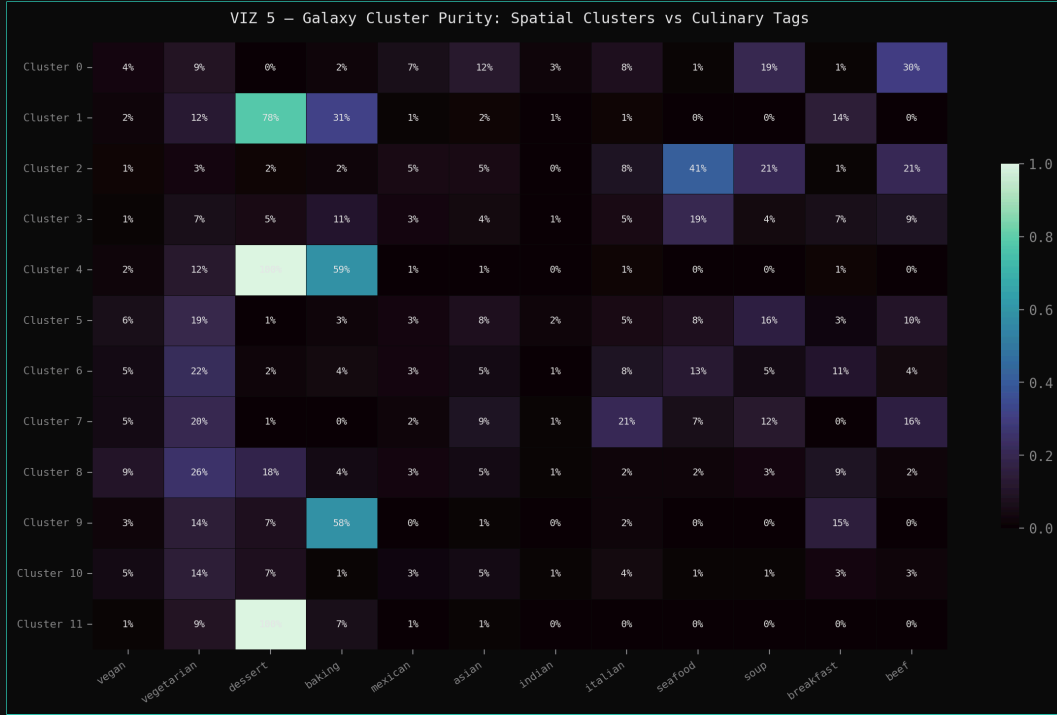


Figure 5: [VIZ 5](#) — Galaxy Cluster Purity Matrix: rows are the 12 K-Means spatial clusters found in 2D UMAP coordinates; columns are culinary tag categories from the Tag Ontology. Each cell shows what fraction of that spatial cluster belongs to each tag. A strong diagonal indicates that the unsupervised geometry is recovering human culinary taxonomy. Off-diagonal noise reflects genuine cross-cuisine overlap rather than model failure.

4. DECISIONS

4.1. UMAP over PCA

While PCA is a common dimensionality reduction technique, it is linear and often fails to capture the complex relationships in high-dimensional text data. UMAP, on the other hand, is non-linear and excels at preserving both local and global structures, making it ideal for creating meaningful clusters in the 3D space. The fidelity comparison in quantifies this preservation directly.

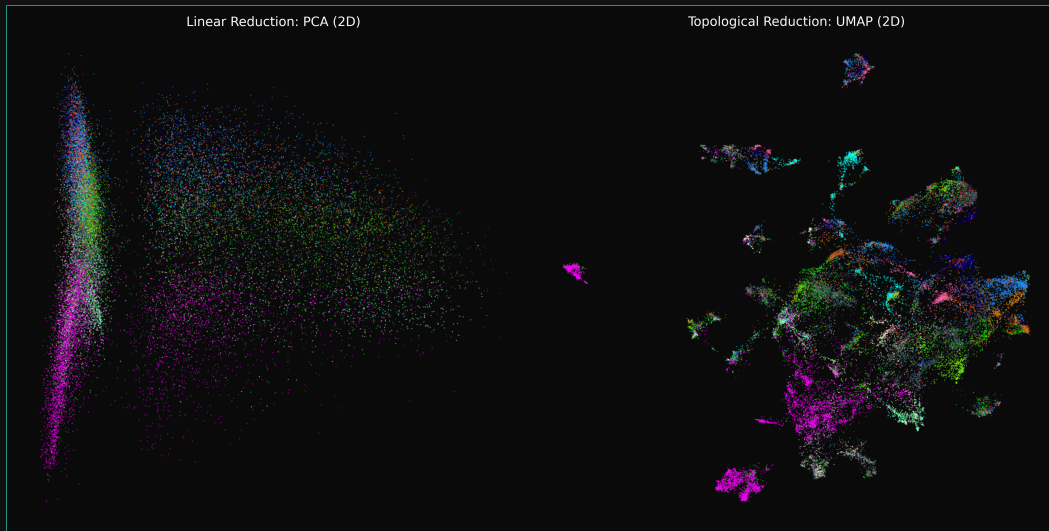


Figure 6: [VIZ 3](#) — A comparison of PCA vs UMAP dimensionality reduction techniques. Each point is a recipe colored by culinary category. The left plot shows PCA coordinates, where clusters are less distinct and categories are more mixed. The right plot shows UMAP coordinates, where clusters are tighter and categories are more visually separable. This demonstrates that UMAP’s non-linear approach better preserves the complex relationships in the data, leading to a more meaningful 3D visualization.

4.2. Custom Ontology for Visualization

Simply plotting the recipes in 3D space would not be visually engaging or informative. To enhance the user experience, a custom ontology was created that categorizes recipes into named clusters (e.g., “Desserts”, “Soups”, “Grilled Meats”) and assigns distinct colors to each cluster. The purity matrix in [Figure 5](#) validates that these color assignments are geometrically grounded — the spatial clusters align with the ontology labels.

4.3. Web Visualization

Rendering this high-dimensional math in a browser required a robust frontend. [Three.js](#) was selected as the modern standard for web-based 3D graphics. While it featured a moderate learning curve, it was essential to achieving the interactable “galaxy” theme envisioned for the project.

4.4. Evaluating Accuracy

A lingering challenge is quantitatively testing the accuracy of the K-NN clusters. While visually the clusters make semantic sense, the purity matrix in [Figure 5](#) represents the first formal attempt at a cluster validity metric. Establishing additional quantitative measures remains a primary focus for future iterations.

5. APPENDIX: TECHNICAL DEFINITIONS

TF-IDF (Term Frequency-Inverse Document Frequency)

An NLP technique that converts text into numerical vectors. It multiplies how often a word appears in a specific document (Term Frequency) by the inverse of how often it appears across all documents (Inverse Document Frequency). In RecipeZ, this naturally penalizes common ingredients like “water” or “salt”, while giving high mathematical weight to unique identifiers like “saffron” or “truffle oil”, allowing for distinct culinary clustering.

Diagnostic: See [Figure 1 — IDF Decay Curve](#) for a visualization of this penalty spectrum across the full vocabulary.

[← Return to Data Pipeline](#)

SVD (Singular Value Decomposition)

A mathematical technique used to reduce data dimensionality by identifying latent features. In this pipeline, it acts as a preprocessing step before UMAP to filter noise and compress the 10,000-feature TF-IDF matrix into 100 dense latent components. The Eckart-Young theorem guarantees that the truncated reconstruction minimizes the Frobenius-norm error among all rank- k approximations.

Diagnostic: See [Figure 6 — Pipeline Trace](#) for how SVD coordinates compare to raw and UMAP coordinates.

[← Return to Data Pipeline](#)

UMAP (Uniform Manifold Approximation and Projection)

A non-linear dimensionality reduction technique. Unlike linear methods like PCA, UMAP preserves both the local relationships (recipes near each other) and the global structure (how the dessert cluster relates to the meat cluster) when compressing high-dimensional vectors into a 3D visual space.

Diagnostic: See [Figure 3 — UMAP Fidelity Witness](#) for a direct measurement of distance preservation before and after compression.

[← Return to Data Pipeline](#)

K-NN (K-Nearest Neighbors)

A spatial search algorithm used here to retrieve the closest recipe vectors to a user’s query. Because TF-IDF creates a vectorized representation of the culinary universe, K-NN measures the cosine similarity from the query vector to all recipe vectors in the database, returning the K closest as results.

Diagnostic: See [Figure 4 — K-NN Query Anatomy](#) for a dissected live query, and [Figure 5 — Cluster Purity Matrix](#) for a spatial cluster validity assessment.

[← Return to Data Pipeline](#)

Three.js

A JavaScript library for rendering 3D graphics in a web browser via WebGL. In RecipEZ, each recipe is rendered as a point in a navigable 3D galaxy, with position determined by UMAP coordinates and color determined by the Tag Ontology cluster. Three.js handles the scene graph, camera, raycasting for click interactions, and instanced geometry for 30,000 simultaneous recipe points.

[← Return to Decisions](#)

Diagnostic Visualizations

The following seven diagnostic plots were generated from the live pipeline to provide empirical evidence that each mathematical stage is functioning as intended.

- [Figure 1](#) — **IDF Decay Curve.** Proves TF-IDF is correctly penalizing ubiquitous ingredients and amplifying rare ones.
- [Figure 3](#) — **UMAP Fidelity Scatter.** Proves UMAP preserves high-dimensional distance rank ordering in the 3D projection.
- [Figure 4](#) — **K-NN Query Anatomy.** Proves K-NN is retrieving semantically coherent neighborhoods for a concrete query.
- [Figure 5](#) — **Tag Purity Matrix.** Proves spatial UMAP clusters correspond to human culinary taxonomy categories.
- [Figure 6](#) — **Pipeline Trace.** Shows the full transformation path of individual recipes across every pipeline stage simultaneously.

Time for a DEMO

TF-IDF

Term Frequency – Inverse Document Frequency

Turning ingredient lists into comparable number arrays.

VECTORIZATION: TF-IDF

Problem: A computer can't compare "Chicken Parm" and "Eggplant Parm" – it can only compare numbers.

The idea: Turn each recipe into a list of numbers – one per ingredient. If an ingredient appears in the recipe, its number goes up.

Recipe A: ["Chicken", "Garlic", "Salt"]

Recipe B: ["Beef", "Garlic", "Salt"]

$$v_A = [\text{Chicken} : 1, \text{Garlic} : 1, \text{Salt} : 1]$$

VECTORIZATION: TF-IDF

Problem: A computer can't compare "Chicken Parm" and "Eggplant Parm" – it can only compare numbers.

The problem with raw counts:

"Salt" is in 99% of recipes. It drowns out everything else. Every recipe ends up pointing in roughly the same direction – toward "Salt."

$$v_A = \left[\text{Chicken : 1, Garlic : 1, } \underbrace{\text{Salt : 1}}_{\text{same in every recipe}} \right]$$

Result: recipes cluster by pantry staples, not by flavor.

TF-IDF: PENALIZING COMMON INGREDIENTS

Fix: scale down ingredients that appear everywhere. Scale up ingredients that are distinctive.

Inverse Document Frequency – the rarity penalty:

$$\text{IDF}(t) = \log \frac{N}{\text{DF}(t)}$$

N = total recipes. $\text{DF}(t)$ = how many recipes contain t .
Rarer ingredient = higher score.

- “Chicken” in 100/10,000 recipes $\rightarrow \text{IDF} = \log(100) = 2.0$
- “Salt” in 9,900/10,000 recipes $\rightarrow \text{IDF} = \log(1.01) \approx 0.004$

TF-IDF: PENALIZING COMMON INGREDIENTS

Fix: scale down ingredients that appear everywhere. Scale up ingredients that are distinctive.

Multiply TF × IDF:

$$v_A = [2.0, \\ 0.5, \\ 0.004]$$

```
>> execution_success
```

“Salt” is nearly zeroed out. “Chicken” dominates.

Two recipes with the same **distinctive** ingredients now point in

the same direction

– and K-NN can find them.

UMAP

Uniform Manifold Approximation and Projection
Compressing 5,000 dimensions into a renderable 3D space.

UMAP: THE DIMENSION PROBLEM

We have 5,000 ingredients. Each one is its own axis. That's 5,000 dimensions.

After TF-IDF, every recipe is a coordinate in $\mathbb{R}^{5000}$.
That's mathematically valid – but completely unrenderable.

$$M \in \mathbb{R}^{10\{,\}000 \times 5\{,\}000} \rightarrow \text{Three.js needs } \mathbb{R}^3$$

We need to compress 5,000 axes down to 3 **without scrambling which recipes are similar to which.**

UMAP: THE DIMENSION PROBLEM

We have 5,000 ingredients. Each one is its own axis. That's 5,000 dimensions.

Why not just drop 4,997 axes?

Any single axis (e.g. "Garlic") tells you almost nothing alone. The **relationships between axes** is where the information lives.

Why not PCA?

PCA finds straight lines of variance. Flavor space is curved – "Ramen" is close to both "Soups" and "Asian Noodles" simultaneously.

We need something that follows the shape of the data.

UMAP: MAPPING THE NEIGHBORHOOD STRUCTURE

UMAP records which recipes are neighbors in 5,000D, then rebuilds those relationships in 3D.

UMAP: MAPPING THE NEIGHBORHOOD STRUCTURE

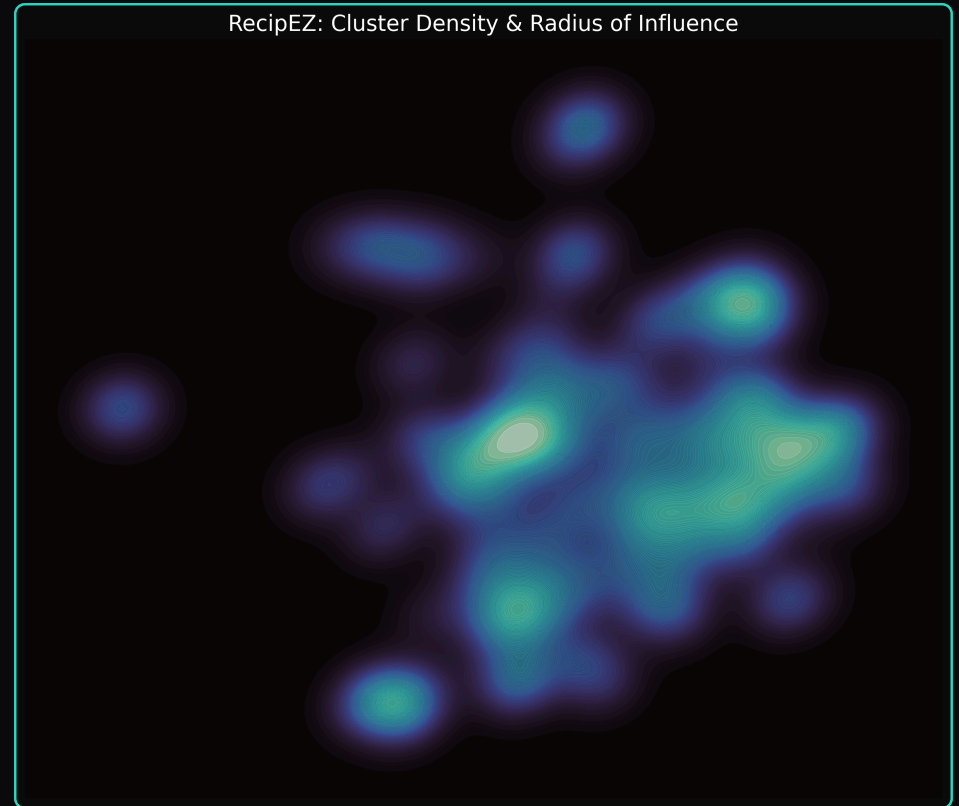
UMAP records which recipes are neighbors in 5,000D, then re-builds those relationships in 3D.

Step 1 – Local neighborhoods

For each recipe, find its k nearest neighbors ($k=15$).

The distance to the closest one sets the local scale ρ_i :

$$\rho_i = \min_{j \in \text{kNN}(i)} d(i, j)$$



UMAP: MAPPING THE NEIGHBORHOOD STRUCTURE

UMAP records which recipes are neighbors in 5,000D, then re-builds those relationships in 3D.

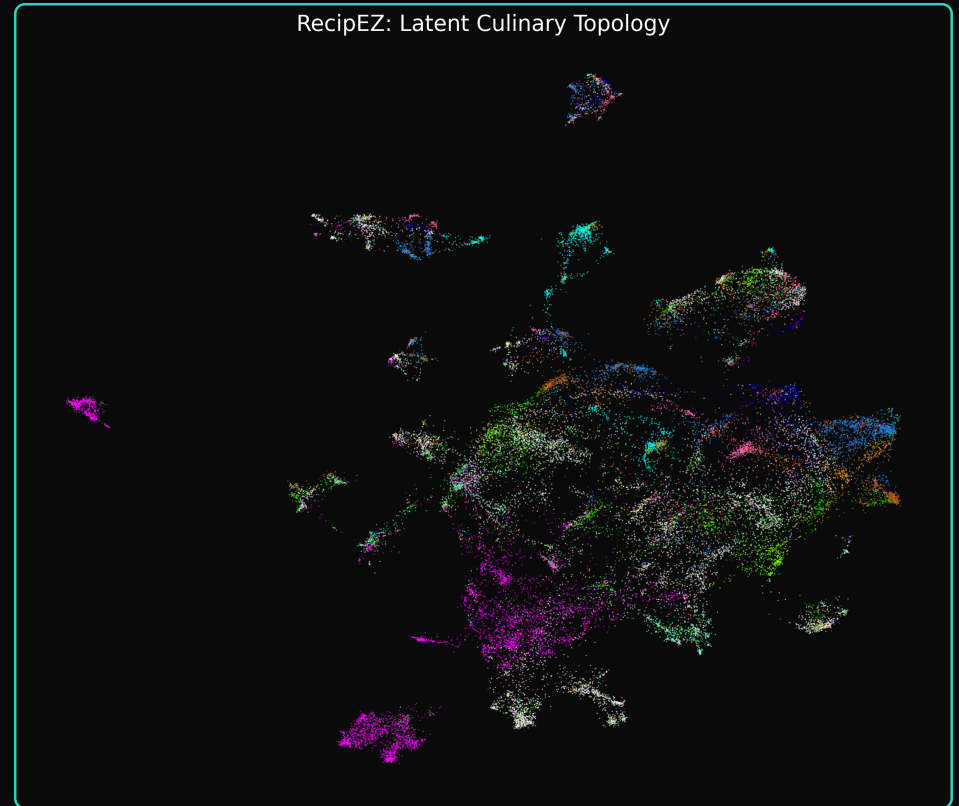
Step 2 – Fuzzy edges

Connections are weighted probabilities, not hard lines:

$$w_{ij} = \exp(-(d(i, j) - \rho_i)/\sigma_i)$$

Nearby recipes: $w \approx 1$

Distant recipes: $w \approx 0$



UMAP: REBUILDING THE MAP IN 3D

Recipes start at random 3D positions. Two forces push and pull them into the right shape.

Attraction – pulls neighbors together

$$F_{\text{att}} = \frac{-2abd^{2b-2}}{1 + ad^{2b}}$$

High- w pairs (similar recipes) get pulled close.

Repulsion – pushes non-neighbors apart

$$F_{\text{rep}} = \frac{2b}{(\varepsilon + d^2)(1 + ad^{2b})}$$

Low- w pairs (unlike recipes) get pushed away.

200 optimization steps. Converges when the 3D map matches the 5,000D map.

Proximity in 3D space = culinary similarity.

UMAP: OUTPUT

Each recipe is now an (x, y, z) coordinate. Similar recipes are physically close.

$$v_{\text{ChickenParm}} = [+14.2, -3.8, +8.1]$$

$$v_{\text{EggplantParm}} = [+13.8, -3.5, +8.4]$$

$$v_{\text{ChocolateCake}} = [-22.1, +17.3, -5.9]$$

"Chicken Parm" ↔ "Eggplant Parm": **0.6 units apart**

"Chicken Parm" ↔ "Chocolate Cake": **37 units apart**

The geometry now reflects culinary reality.

UMAP: OUTPUT

Each recipe is now an (x,y,z) coordinate. Similar recipes are physically close.

```
>> render_ready
```

5,000 ingredient axes → 3 spatial coordinates.

Three.js can place every recipe as a point in a navigable 3D galaxy.

TF-IDF ✓ → UMAP ✓ → SVD → K-NN

SVD

Singular Value Decomposition

Discovering hidden flavor dimensions from ingredient co-occurrence.

SVD: FINDING HIDDEN FLAVOR GROUPS

TF-IDF treats every ingredient as independent. But ingredients travel in packs.

Basil, Oregano, Tomato, and Mozzarella almost always appear together.

They're not four separate axes of information – they're one concept: **Italian**.

TF-IDF has no way to know this. The 5,000D space is full of redundant, correlated dimensions.

Consequence: K-NN is comparing noise alongside signal.

SVD: FINDING HIDDEN FLAVOR GROUPS

TF-IDF treats every ingredient as independent. But ingredients travel in packs.

SVD discovers these “flavor axes” automatically by analyzing which ingredients co-occur across the whole corpus.

$$M_{10\{\},5\{\}} \rightarrow \text{SVD} \rightarrow \tilde{M}_{10\{\},128}$$

5,000 raw ingredient axes $\rightarrow$ 128 **latent flavor dimensions**.
91% of the information. 2.5% of the dimensions.

SVD: THE DECOMPOSITION

Any matrix can be factored into three simpler matrices.

$$M = U\Sigma V^T$$

U

Shape: $N \times N$

Each row = one recipe as a mix of flavor dimensions.

$$U^T U = I$$

Σ

Diagonal.

$$\sigma_1 \geq \sigma_2 \geq \dots \geq 0$$

Each value = how much variance that flavor dimension explains.

Sorted by importance.

V^T

Shape: $D \times D$

Each row = the ingredient blend for one flavor dimension.

$$V V^T = I$$

SVD: KEEPING ONLY WHAT MATTERS

Keep the top k dimensions. Drop the rest – they're noise.

$$M \approx \tilde{M} = U_k \Sigma_k V_k^T$$

$$\|M - \tilde{M}\|_F = \sqrt{\sum_{i=k+1}^r \sigma_i^2} \rightarrow \text{minimized}$$

The Eckart-Young theorem guarantees this is the **best possible** rank- k approximation.

At $k=128$: variance explained = **91.3%**, dimensions dropped = **97.4%**.

The small σ_i values we discard encode coincidences – not real culinary relationships.
We keep the signal. We discard the noise.

SVD: WHAT THE FLAVOR DIMENSIONS LOOK LIKE

The rows of V_k^T are not hand-labeled – the algorithm discovers these groupings on its own.

Dim 1 ($\sigma=142$): Basil +0.41 · Tomato +0.38 · Oregano +0.35 · Mozzarella +0.29
→ Italian / Mediterranean

Dim 2 ($\sigma=119$): Soy Sauce +0.44 · Ginger +0.39 · Sesame +0.31 · Rice Wine +0.28
→ East Asian

Dim 3 ($\sigma=97$): Cumin +0.42 · Chili +0.40 · Coriander +0.33 · Lime +0.27
→ South Asian / Mexican

Dim 4 ($\sigma=84$): Butter +0.45 · Sugar +0.43 · Vanilla +0.38 · Flour +0.35
→ Desserts / Baking

ChickenParm: Dim1=0.91 · Dim2=0.04 · Dim3=0.07 · Dim4=0.02

SVD: OUTPUT

Input:

M_sparse shape=(10000, 5000) nnz=198,421 density=0.4%

Fit — randomized SVD, $k = 128$:

variance_explained = 91.3%

top singular values: [142.3, 118.7, 97.1, 84.2, ...]

Output:

M_latent shape=(10000, 128) dtype=float32 density=100%

>> execution_success

TF-IDF ✓ → UMAP ✓ → SVD ✓ → K-NN

K-NEAREST NEIGHBORS

K-Nearest Neighbors

Efficiently finding the closest recipes in vector space.

K-NN: FINDING SIMILAR RECIPES

Every recipe is now a 128D coordinate. “Similar recipes” means “nearby coordinates.”

A user clicks “Chicken Parmesan.”

We need the K recipes whose vectors sit closest to $\tilde{v}_{\text{ChickenParm}}$.

Brute-force: compare the query against every recipe.

$$\text{Cost} = O(N \cdot D) = O(10\{, \}000 \times 128) = 1.28 \times 10^6 \text{ ops/query}$$

Fast enough now — but at 1M recipes $\times$ raw 5K dims: 5×10^9 ops.

We need a smarter structure.

K-NN: FINDING SIMILAR RECIPES

Every recipe is now a 128D coordinate. “Similar recipes” means “nearby coordinates.”

The insight: if we spatially index the vectors, we can skip most comparisons.

Recipes that are far away geometrically **can never be close neighbors**

– so we don’t check them.

We use a **KD-Tree** – a binary tree that partitions vector space into nested regions.

Build once. Every query becomes $O(\log N)$ instead of $O(N)$.

KD-TREE: SPLITTING SPACE

Recursively cuts the search space in half along the most spread-out dimension.

At each node, split at the median of the widest dimension:

$$\pi_{d,t} = \{x \in \mathbb{R}^D : x_d = t\}, \quad d = \arg \max_j \text{Var}(x_j)$$

Left: $x_d < t$. Right: $x_d \geq t$. Recurse until leaves hold ≤ 40 recipes.

Build cost: $O(DN \log N)$ – paid once.

For 10,000 recipes: 14 branch decisions to reach a leaf.

KD-TREE: SPLITTING SPACE

Recursively cuts the search space in half along the most spread-out dimension.

Pruning — the key trick:

After checking the nearest leaf, walk back up the tree.

At each branch, measure the distance to the splitting plane:

$$d_{\pi} = |\tilde{v}_{\text{query}}[d] - t|$$

If $d_{\pi} \geq$ current best distance: **skip the entire subtree.**

For clustered recipe data: **96% of the tree is pruned per query.**
312 nodes visited out of 10,000.

K-NN: DISTANCE METRIC

We use cosine similarity – it measures the angle between vectors, not their length.

Euclidean – measures gap

$$d = \sqrt{\sum_i (A_i - B_i)^2}$$

A recipe scaled to double portions looks far from its twin despite identical flavor.

Fooled by scale.

Cosine – measures angle

$$\text{sim}(A, B) = \frac{A \cdot B}{\|A\| \cdot \|B\|}$$

Same flavor ratios = 1.0 regardless of recipe size.

Scale-invariant.

L2-normalize all vectors: $\hat{v} = v / \|v\|$.

On the unit sphere, cosine similarity equals Euclidean distance.

One step – best of both metrics.

K-NN: OUTPUT

```
find_neighbors(ChickenParm, K=5, metric=cosine)
```

[cos=0.97]	Eggplant Parmesan	→	[13.8, -3.5, 8.4, ...]
[cos=0.94]	Chicken Milanese	→	[14.9, -4.1, 7.6, ...]
[cos=0.91]	Veal Parmesan	→	[13.1, -3.2, 9.0, ...]
[cos=0.88]	Baked Ziti	→	[12.4, -5.0, 8.8, ...]
[cos=0.85]	Chicken Marsala	→	[15.3, -2.9, 7.1, ...]

K-NN: OUTPUT

```
find_neighbors(ChickenParm, K=5, metric=cosine)
```

```
>> query_success
```

```
latency: ~2ms | nodes_visited: 312 / 10,000 | pruned: 96.9%
```

TF-IDF ✓ → UMAP ✓ → SVD ✓ → K-NN ✓

SBERT

Sentence-BERT

A second vectorization strategy – this time, biased toward meaning.

SBERT: A SECOND VECTORIZER

TF-IDF vectorizes by token frequency. SBERT is a completely separate model that vectorizes by meaning.

Both produce vectors that feed into the same UMAP visualization pipeline.

They are two different answers to the same question:

"How do we turn a recipe into a point in space?"

TF-IDF

Counts ingredient tokens.

"Chicken" and "Poultry"

are unrelated axes.

Lexical.

SBERT

Encodes semantic meaning.

"Chicken" and "Poultry"

land near each other.

Semantic.

SBERT: A SECOND VECTORIZER

TF-IDF vectorizes by token frequency. SBERT is a completely separate model that vectorizes by meaning.

Why have both?

They produce different 3D galaxies — different lenses on the same data.

TF-IDF clusters by **shared ingredients**.

SBERT clusters by **culinary meaning and description**.

A recipe called "Braised Short Ribs" and one called "Slow-Cooked Beef in Red Wine"

will be far apart in TF-IDF space — and close together in

SBERT: HOW IT WORKS

SBERT is a fine-tuned BERT model that outputs a single fixed-size vector per sentence.

Standard BERT outputs one vector **per token**, not per sentence. To get a single sentence vector, we average all token outputs – **mean pooling**:

$$u = \frac{1}{n} \sum_{i=1}^n h_i \in \mathbb{R}^{768}$$

Where h_i is the output for the i -th token.
Always 768 dimensions, regardless of input length.

SBERT: HOW IT WORKS

SBERT is a fine-tuned BERT model that outputs a single fixed-size vector per sentence.

How meaning is trained in:

SBERT is trained with triplet loss – similar sentences are pulled together,
dissimilar ones pushed apart:

$$\mathcal{L} = \max(0, \|u_a - u_p\|^2 - \|u_a - u_n\|^2 + \varepsilon)$$

After training: **geometric distance $\approx$ semantic distance.**

“Spicy chicken stir-fry” and “fiery poultry wok dish” land close together.

Same concept $\rightarrow$ small angle $\rightarrow$ high cosine similarity.

SBERT: THREE VISUALIZATION MODES

The site runs three separate SBERT models in parallel, each producing a different 3D galaxy.

Ingredients

Mode A

SBERT encodes the **ingredient list** as a sentence.

Clusters by what's in the recipe.

Similar to TF-IDF but meaning-aware –

Recipe Name

Mode B

SBERT encodes only the **recipe title**.

Clusters by dish concept and cuisine.

"Chicken Tikka" and "Chicken Curry"

Combined

Mode C

SBERT encodes **name + ingredients** together.

Balances both signals.

The default view – captures both what a

"olive oil" and cluster even if in- dish **is** and what it
"EVOO" land to- gredients differ. **contains.**
gether.

SBERT: SAME PIPELINE, DIFFERENT LENS

Each SBERT mode feeds directly into the same UMAP → Three.js pipeline as TF-IDF.

All three SBERT models output 768D vectors.

UMAP compresses each set independently into its own 3D coordinates.

Mode A (Ingredients): `v_ChickenParm` = [+11.3, -2.1, +6.8]

Mode B (Name): `v_ChickenParm` = [+8.7, +4.2, -1.3]

Mode C (Combined): `v_ChickenParm` = [+12.9, -1.7, +5.2]

Three different maps. Same recipes. Different groupings depending

on which signal matters most to the user.

SBERT: SAME PIPELINE, DIFFERENT LENS

Each SBERT mode feeds directly into the same UMAP → Three.js pipeline as TF-IDF.

```
>> all_modes_ready
```

TF-IDF vectorizer	→	UMAP	→	3D galaxy (token frequency)
SBERT / Ingredients	→	UMAP	→	3D galaxy (ingredient meaning)
SBERT / Name	→	UMAP	→	3D galaxy (dish concept)
SBERT / Combined	→	UMAP	→	3D galaxy (full context)

The user selects a mode. The visualization updates.
Same K-NN search engine underneath — only the vector space changes.

K-NN EVALUATION

[Returning to the primary text...](#)